

## ASSIGNMENT - CHOWLY (BUILD)

In the last assignment you were hired as the Software Engineer for Chowly and you produced an engineered model for the solution. A model, however, does not serve a single customer. Restaurants do not pay for tables on paper, they pay for software that works.

Chowly has now approved your model. The restaurant opens its doors shortly and the platform must be live, deployed and usable before the first customer walks in.

Also, no serious engineer builds alone today. AI is now part of the toolchain of every top class engineering team, and you are expected to work with it.

### Features

A visiting customer can view the restaurant's food and drinks menus and place an order. The waiting time for each order can be viewed by the customer along with other order details.

After a customer submits an order, the order is assigned to a waiter who updates the order details by specifying the chef and the bartender who prepared the order on the platform.

In a case when there is serious delay in the preparation of the order, the customer can submit a complaint and give a low rating on that order.

Most importantly, payment for the order is made on the platform just before the customer exits the restaurant.

### Requirements

- 1 The menu.** Food and drinks, loaded into your database by you. Each item carries a name, a price and a preparation time.
- 2 Placing an order.** A customer selects items, submits the order, and is shown the order details and the waiting time for that order.
- 3 Assigning the order.** A waiter sees the orders, opens one, records the chef and the bartender who prepared it from a staff list you loaded yourself, and marks the order as served.
- 4 Complaint and rating.** A customer submits a complaint and a rating on an order, and both are stored against that order.

- 5 **Payment.** A button that records the payment and marks the order as paid. The payment may be pretend, but it must be recorded and it must be clearly labelled as pretend.
- 6 **The two roles.** A way to act as the customer and as the waiter. Logins are not required, a simple switch is enough.
- 7 **Real storage.** Everything above is saved in a database. If the page is refreshed, the data is still there.
- 8 **A live link.** The application is deployed and anybody with the link can use it.

## Task

- Use any tech stack you are comfortable with. You will not be marked on the stack you chose, you will be marked on the solution it produced.
- Build on top of the data model you designed. Where the build forces you to change the model, change it and say why in your document.
- You must use AI to facilitate your work, and you must be able to show how you used it.

## Bonus

Bonus points will be added for anything you build beyond the requirements, and for an application that is beautifully designed and well presented.

## Deliverables

- 1 **The git repository of your codebase.** The link must be accessible to your facilitators, and the commit history must show the work as it was done.
- 2 **The URL of the deployed application.** It must open, and it must work.
- 3 **A document describing the application.** The document must cover:
  - How you built it - the stack, the structure, the data model as it was finally implemented, and how the application was deployed.
  - How you used AI - the tools, what you asked them to do, what you accepted, what you rejected and what you had to correct yourself.
  - The specific behaviour of the application - what happens at every step of the story, from menu to payment.
  - How to use it - a walkthrough a stranger can follow on your deployed link, including how to switch between the customer and the waiter.

Ensure you describe the behaviour of every feature you implement.

For example:

**Menu browsing** - a customer opens the app at a table and views the food and drinks currently available

**Order placement** - a customer selects items and submits an order, and is shown the waiting time and the other details of that order

**Order assignment** - the order is assigned to a waiter, who records the chef and the bartender that prepared it

**Complaint and rating** - where an order is delayed, the customer submits a complaint and gives a low rating against that order

**Payment** - the customer pays for the order on the platform just before exiting the restaurant

---

Ensure your application is **COMPLETE, DEPLOYED** and takes into the consideration the entire **STORY !**